# Deconstructing Clean Architecture in Go

## From Blueprint to Running Code: A Tour of a Production-Ready CRUD Application



Clean Architecture

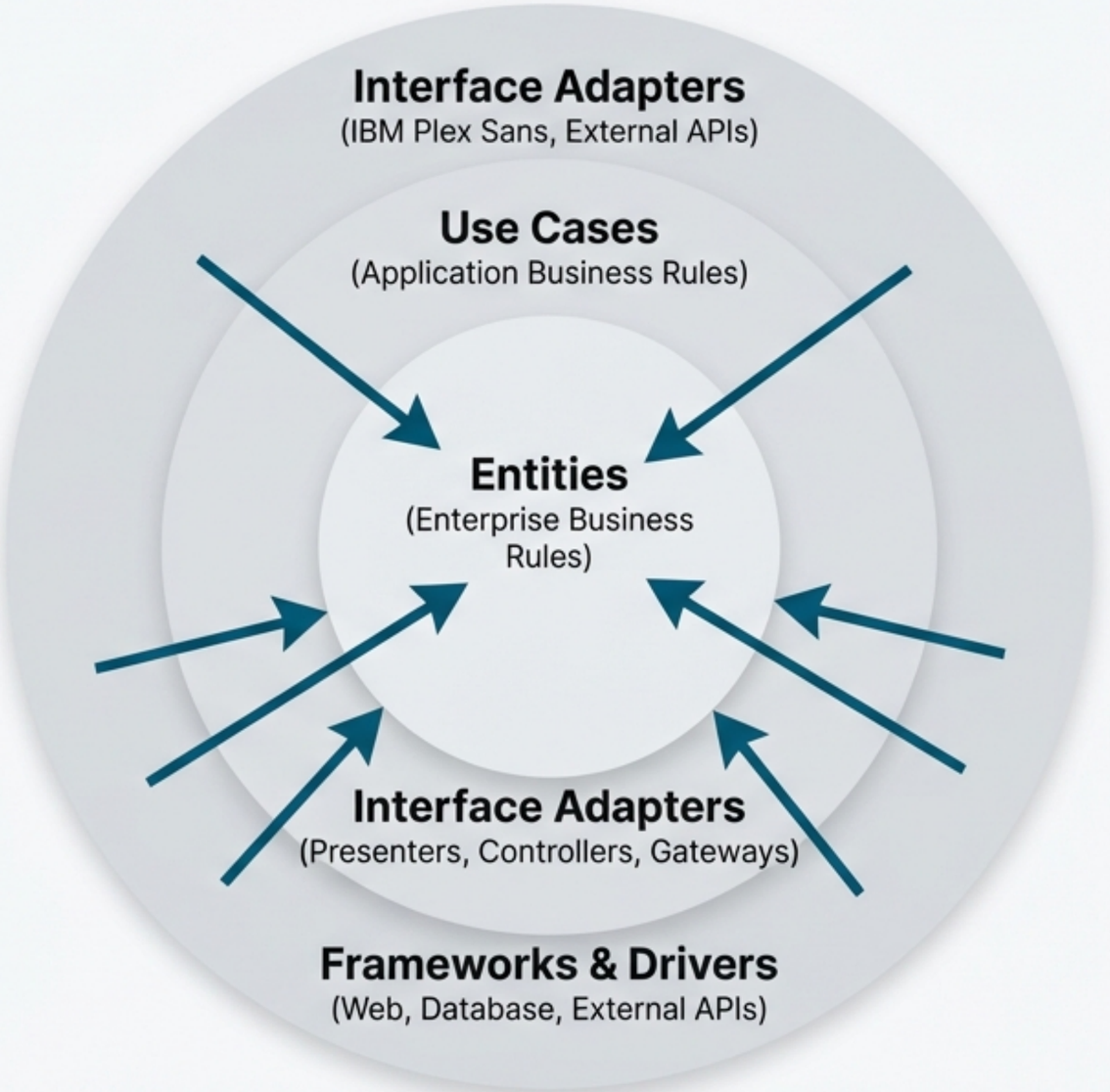
OpenAPI First

Dockerized

Modern Tooling

# The Core Philosophy: The Dependency Rule

Clean Architecture organizes code into concentric layers, with business rules at the center and technical details at the outer layers. The fundamental rule is that dependencies can only point inward.



# The Payoff: Why This Architecture Matters

## 1.
### Independence from Frameworks

Your core business rules are not tied to any external libraries or frameworks.

## 2.
### Supreme Testability

Business logic can be tested in isolation, without needing a UI, database, or external services.

## 3.
### Independence from the UI

The user interface can be changed or replaced entirely without affecting the core system.

## 4.
### Independence from the Database

Your business rules are not **bound to a specific database**. Swap from MySQL to Postgres with minimal impact on the core logic.

## 5.
### Independence from External Services

The core application is ignorant of the outside world, making it robust and adaptable.

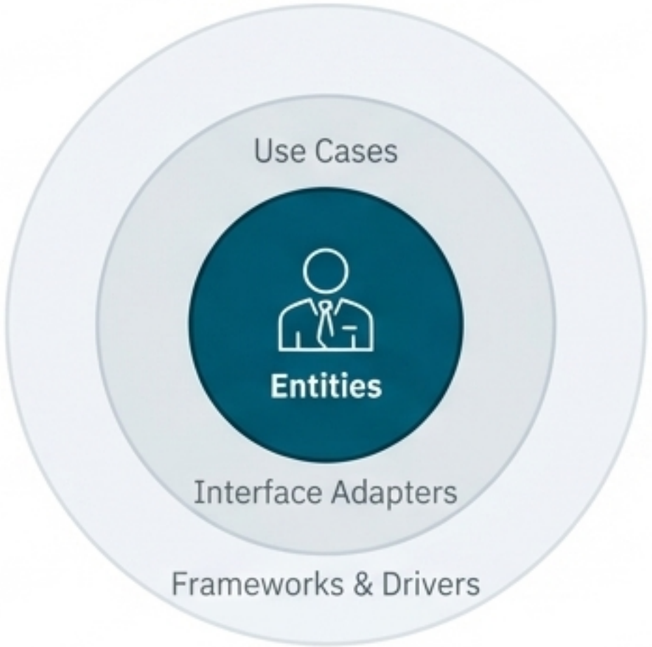
# The Blueprint: Project Structure

```
.
├── cmd/api/                # Application entry point
├── internal/
│   ├── domain/             # Entities & repository interfaces
│   ├── usecase/            # Business logic
│   ├── dto/                # Data Transfer Objects
│   ├── mapper/             # Object mapping
│   ├── presenter/          # Data formatting
│   ├── api/                # HTTP server implementation
│   ├── infrastructure/     # Database implementation
│   └── delivery/           # Route configuration
├── docker/                 # Container configuration
├── Dockerfile
├── docker-compose.yml
└── oapi.yaml               # OpenAPI 3.0 specification
```

# Layer 1: The Domain (Enterprise Business Rules)

**Purpose**: Contains the core business entities and the interfaces for data access (repositories). This layer has zero external dependencies.

**Key Components**:
- Business models (`User` struct)
- Repository interfaces (`UserRepository`)



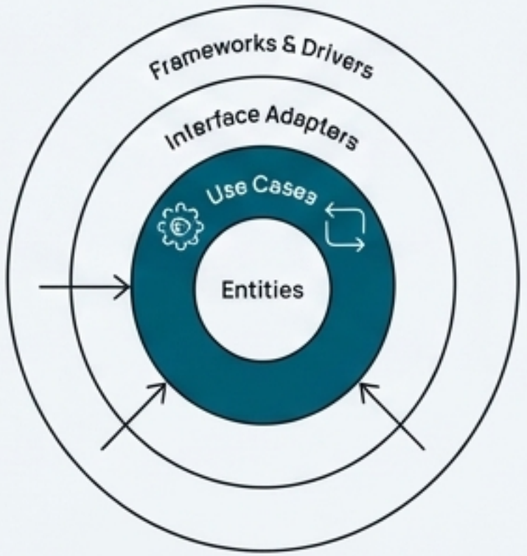

```go
// internal/domain/user.go

// The core data structure, independent of any framework.
type User struct {
    ID        int       `json:"id"`
    Name      string    `json:"name"`
    Email     string    `json:"email"`
    CreatedAt time.Time `json:"created_at"`
    UpdatedAt time.Time `json:"updated_at"`
}

// The contract for data persistence. The domain doesn't
// know or care how this is implemented (SQL, NoSQL, etc).
type UserRepository interface {
    Create(user *User) error
    GetByID(id int) (*User, error)
    GetAll() ([]*User, error)
    Update(user *User) error
    Delete(id int) error
}
```

# Layer 2: The Use Case (Application Business Rules)

**Purpose:** Implements application-specific business rules. It orchestrates the flow of data to and from the Domain layer to achieve business objectives.

**Key Components:** Application-specific logic, validation, and error handling.

**Dependencies:** Domain, DTO, Mapper, Presenter




```go
func (u *UserUsecase) CreateUser(name, email string)
([]byte, error) {
    // 1. Basic validation
    if name == "" || email == "" {
        return u.presenter.PresentError("name and email are
    required")
    }

    // 2. Map request data to a domain object
    req := &dto.CreateUserRequest{Name: name, Email:
    email}
    domainUser := mapper.MapCreateRequestToDomain(req)

    // 3. Use the repository interface to persist the
    domain object
    if err := u.userRepo.Create(domainUser); err != nil {
        return u.presenter.PresentError(err.Error())
    }

    // 4. Present the successful result
    return u.presenter.PresentUser(domainUser)
}
```

# Layers 3-5: Adapters for Data Transformation

**Purpose**: These layers manage the conversion of data between different formats required by the core application and the external world (e.g., HTTP requests/responses).

## Column 1: DTO (`internal/dto/user_dto.go`)

Defines structures for API requests and responses.

```
type CreateUserRequest struct {
    Name  string `json:"name" example:"John Doe"`
    Email string `json:"email" example:"john@example.com"`
}
```

## Column 2: Mapper (`internal/mapper/user_mapper.go`)

Handles object transformation between layers using `jinzhu/copier`.

```
func MapCreateRequestToDomain(req *dto.CreateUserRequest) *domain.User {
    user := &domain.User{}
    copier.Copy(user, req) // Automatic field mapping
    user.CreatedAt = time.Now()
    user.UpdatedAt = time.Now()
    return user
}
```

## Column 3: Presenter (`internal/presenter/user_presenter.go`)

Formats final output (e.g., to JSON).

```
func (p *HTTPUserPresenter) PresentUser(user *domain.User) ([]byte, error) {
    response := mapper.MapDomainToResponse(user)
    return json.Marshal(response)
}
```

# Layers 6-8: Frameworks & Drivers

**Purpose**: The outermost layers contain all specific implementations for external concerns like databases, web frameworks, and routing. They are the most volatile and easiest to change.

## Infrastructure (`internal/infrastructure/database/user_repository.go`)

The concrete implementation of the `UserRepository` interface.

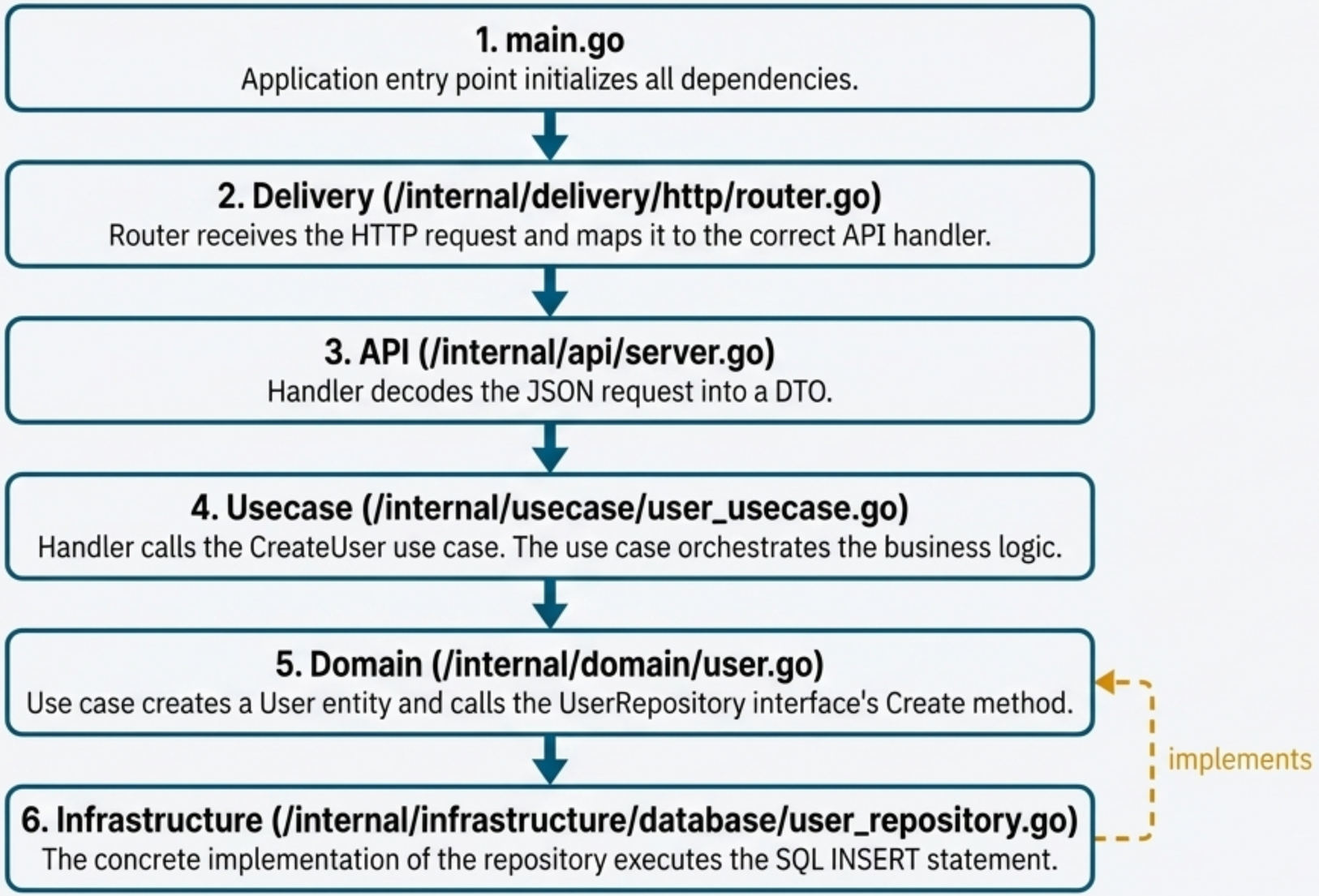
```
// This struct holds the actual database connection.
type MySQLUserRepository struct {
    db *sql.DB
}

// This is the concrete implementation of the domain interface.
func (r *MySQLUserRepository) Create(user *domain.User) error {
    query := "INSERT INTO users (name, email, created_at,
updated_at) VALUES (?, ?, ?, ?)"
    result, err := r.db.Exec(query, user.Name, user.Email,
user.CreatedAt, user.UpdatedAt)
    // ... implementation details ...
    return nil
}
```

## API & Delivery (`internal/api/server.go`, `internal/delivery/http/router.go`)

The HTTP server and router that handle incoming requests and delegate to the use cases.

```
// The router connects HTTP paths and methods to handlers.
func (r *Router) SetupRoutes() *http.ServeMux {
    mux := http.NewServeMux()
    mux.HandleFunc("/users", func(w http.ResponseWriter, req
*http.Request) {
        switch req.Method {
        case http.MethodPost:
            r.server.CreateUser(w, req) // Delegates to the
handler
        // ... other methods ...
        }
    })
    return mux
}
```

# Anatomy of a Request: The Dependency Flow in Action



# Modern Tooling I: Specification-First with OpenAPI

The API is defined in `oapi.yaml` before implementation, serving as a single source of truth for documentation, validation, and even code generation.

**Benefits**:
- Complete API Documentation
- Standard Compliance (OpenAPI 3.0)
- Code Generation Ready (`oapi-codegen`)

```yaml
paths:
  /users:
    post:
      summary: Create a new user
      requestBody:
        required: true
        content:
          application/json:
            schema:
              $ref: '#/components/schemas/CreateUserRequest'
      responses:
        '201':
          description: User created successfully
          content:
           application/json:
             schema:
               $ref: '#/components/schemas/UserResponse'
```

## **API Endpoints**

- `POST /users` - Create new user
- `GET /users` - List all users
- `GET /user?id={id}` - Get user by ID
- `PUT /user?id={id}` - Update user
- `DELETE /user?id={id}` - Delete user

# Modern Tooling II: Automated Object Mapping

The project uses `jinzhu/copier` to eliminate tedious and error-prone manual field assignments when mapping between DTOs and Domain entities.

**Key Features**:
- Automatic field copying
- Type safety
- Performance optimized

**Before (Manual & Error-Prone)**

internal/mapper/user_mapper.go

```
// return &domain.User{
//     Name: req.Name,
//     Email: req.Email,
// }
```

**After (Concise & Maintainable)**

JetBrains Mono

```
user := &domain.User{}
copier.Copy(user, req)
return user
```

# Modern Tooling III: DRY Principles in the API Layer

The API layer uses centralized helper methods to eliminate code duplication, ensuring consistent responses and easier maintenance.

## Reusable Helper Methods

`writeJSONResponse()`
Centralized JSON response writing.

`writeErrorResponse()`
Standardized error response handling.

`extractAndValidateID()`
Reusable ID extraction and validation.

`decodeJSONBody()`
Reusable JSON decoding and validation.

## Tangible Benefits

**Reduced Code Duplication**

From **~200 lines** to **~120 lines** per handler.

**Consistent Responses**

All endpoints use the same success and error patterns.

**Easier Maintenance**

A change to the response format only requires an update in one place.

# Get Started in 60 Seconds: The Docker Workflow

**Prerequisites**: Docker and Docker Compose

## 1
### Clone

```
git clone <repository-url>
```

## 2
### Change Directory

```
cd golang-clean-architecture
```

## 3
### Run

```
docker-compose up --build
```

## What Happens Next

- Builds and starts the MySQL database container.
- Builds and starts the Go application container.
- The API becomes available at `http://localhost:8080`.

## Key Docker Commands

View Logs: `docker-compose logs -f api`

Stop Services: `docker-compose down`

Clean Everything: `docker-compose down -v`

# Interacting with the Live API

Once running, you can interact with all API endpoints using any HTTP client.

### Create a User (`POST`)

```
curl -X POST http://localhost:8080/users \
    -H "Content-Type: application/json" \
    -d '{"name": "John Doe", "email": "john@example.com"}'
```

### Get All Users (`GET`)

```
curl http://localhost:8080/users
```

### Get User by ID (`GET`)

```
curl "http://localhost:8080/user?id=1"
```

### Delete User (`DELETE`)

```
curl -X DELETE "http://localhost:8080/user?id=1"
```

# A Blueprint for Modern Go Applications

## Key Architectural Principles Embodied

- **Single Responsibility:** Each component has one reason to change.
- **Dependency Inversion:** High-level modules do not depend on low-level modules; both depend on abstractions.
- **Separation of Concerns:** Each layer handles a specific, isolated concern.
- **Testability by Design:** All components can be tested in isolation.
- **Specification-First Development:** The API is defined before it is implemented.

## Technology Stack at a Glance

**Language:** Go (using only the standard library for HTTP)

**Database:** MySQL 8.0

**Containerization:** Docker & Docker Compose

**API Specification:** OpenAPI 3.0

**Tooling:** `jinzhu/copier`, `oapi-codegen`